IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0000000

An exploration of controllability in symbolic music infilling

RUI GUO[1] DORIEN HERREMANS[2],
[1]University of Sussex, Brighton, UK (e-mail: r.guo@sussex.ac.uk)
[2]Singapore University of Technology and Design, Singapore (e-mail: dorien_herremans@sutd.edu.sg)
Corresponding author: Rui Guo (e-mail: r.guo@sussex.ac.uk).

ABSTRACT This study uses a transformer model to enhance the controllability of generative symbolic music models, specifically related to the infilling task. We introduce a novel Symbolic Music representation with Explicit Rest notation (SMER) encoding incorporating five basic duration types and explicit rest note tokens similar to standard music notation. We compare this approach with another event-based symbolic music encoding called ''REMI'' (REvamped MIDI-derived events) regarding controllability over bar-level tension and track-level texture, which refers to how musical elements such as melody and harmony are combined in a musical composition. The SMER encoding is compared with another controllable infilling model, Multi-Track Music Machine (MMM), for track-level density controllability. The findings confirm that the proposed SMER demonstrates superior controllability and generates music stylistically more similar to the original music than that generated by MMM. We propose strategies to further enhance track-level texture control by training two models, controlling each bar's texture (SMER BAR), and predicting each bar's texture after each bar's generation (SMER Pre). Those two models with bar-level texture control effectively increase track-level texture control. To explore the interaction of the controllability of different controls, we thoroughly analyzed the controllability of different types and levels of texture controls. Finally, we implemented an interactive interface to facilitate interactive music composition with AI to help bridge the gap between the AI model and musicians.

INDEX TERMS symbolic music generation, conditional sequence generation, music encodings, interactions of controls

I. INTRODUCTION

The growing abilities of machine learning techniques has enabled artificial intelligence (AI) music generation systems to generate longer music sequences [1, 2]. However, at the moment, the ''knobs'' users can control on those systems are insufficient to achieve true co-creation of AI and human users. Feedback from the AI Song Contest 2020 participants highlighted that many AI systems were not steerable, context-aware, or decomposable, making true co-creation between AI and human users challenging [3]. Users frequently have to ''reverse engineer the process by priming the model's input sequence with music that contains the desired properties'' [3], which is inefficient and time-consuming.

A significant gap in the current AI music generation is the lack of controllability, especially at different levels, such as bars and tracks. Existing models often do not allow users to specify detailed musical attributes, forcing them to generate multiple fragments and select the best fit for their needs. The lack of controllability hinders the creative process and limits the practical application of AI in music composition.

To address these limitations, we focus on improving controllability in the music infilling task, where the model fills in missing parts of a musical piece based on the surrounding context. The missing parts can range from individual notes to entire tracks or bars. Controllability in this context refers to the model's ability to generate music that adheres closely to the parameters set by the users. Enhancing controllability allows users to create content that aligns directly with their creative ideas without repetitive generation and selection.

Our first contribution is the introduction of a novel symbolic music encoding called SMER (Symbolic Music representation with Explicit Rest notation) (Step 1 in Figure 1). We hypothesize that a more musically meaningful encoding can facilitate better control over the generated music. Unlike the widely used REMI encoding format [4], which uses implicit rest tokens, SMER explicitly represents

rests, providing a clearer structural representation of the music. This explicit notation can help the model understand musical timing and silence more effectively, improving controllability at both bar and track levels. We perform an extensive experiment to evaluate the SMER and REMI-based generated models regarding bar-level and track-level controllability.

Building upon SMER, our second contribution is the incorporation of bar-level texture control tokens, resulting in the ''SMER BAR'' model (Step 2 in Figure 1) and ''SMER Pre'' model (Step 3 in Figure 1). By adding bar-level texture controls, we provide the model with detailed information about the desired musical attributes within each bar, which is particularly beneficial for infilling tasks where only specific bars or tracks need to be regenerated. This fine-grained control ensures that the newly generated material aligns closely with user-specified density, occupation, or polyphony levels in the targeted region, without altering other parts of the piece. Additionally, we introduce an "unk" token that can selectively relax constraints on certain bar-level textures, striking a balance between strict enforcement (e.g., matching a precise density) and creative flexibility (e.g., leaving each bar's density unconstrained).

Furthermore, we explore the model's predictive capability regarding bar-level textures. In the "SMER Pre" model, bar-level textures are predicted before generating each bar and then used as autoregressive inputs to guide the note generation. By doing so, the model can dynamically adjust subsequent bars in line with both predicted and user-specified bar-level textures, fostering consistency across the reconstructed section. This approach leverages the interaction between hierarchical control levels to fine-tune each generation step more effectively.

In addition, we explore the interactions between different texture controls. When more texture parameters are added to the model, the other parameters may affect each parameter's controllability. Hence, in an extensive experiment, we analyze the interaction of different types of texture controls and levels of texture controls.

Finally, SMER-Pre is compared to another infilling model, Multi-Track Music Machine (MMM)[5], both objectively and subjectively. Several pitch and duration metrics are compared to evaluate stylistic similarity to the original music, i.e., if the generated music changes but is still stylishly similar to the original music. The controllability of the track-level density is also compared between the two models. Finally, a user listening test with 24 participants with different musical backgrounds is performed to evaluate the music quality subjectively. The results prove a superior track-level texture control and better music quality.

An Ableton Live plugin is developed to facilitate interactive music composition between AI and humans. Musicians can use the plugin to compose music interactively and iteratively. The model can be hosted locally and remotely, and the model code and the plugin are shared online at https://github.com/ruiguo-bio/smer_music_generation.git..

In conclusion, our contributions are as follows:

1) **SMER Encoding**: We introduce SMER, a new symbolic music representation with explicit rest notation. This encoding enhances the model's understanding of musical structure, improving controllability compared to existing encodings like REMI.
2) **Enhanced Texture Control with Bar-Level Texture Tokens**: Building upon SMER, we incorporate bar-level texture control tokens in the ''SMER BAR'' and ''SMER Pre'' models. Adding and predicting bar-level textures enables the model to adjust generations based on fine-grained controls, enhancing track-level texture control and adherence to user-specified attributes.
3) **Analysis of Control Interactions**: We extensively analyze the interactions between different types and levels of texture controls. Our findings demonstrate that fine-grained control at the bar level positively impacts controllability at the track level, addressing the gap in understanding the relationship between hierarchical control levels.
4) **Interactive Composition Tool**: We developed an Ableton Live plugin to bridge the gap between AI models and musicians. This tool provides an interface for interactive and iterative music composition, facilitating the practical application of our models in real-world creative workflows.

II. CONTROLLABLE SYMBOLIC MUSIC GENERATION

Controllable generation systems empower users to steer parameters during music creation, including genre, style, emotion, key, and other musical properties such as track-level density. This section provides an overview of control features used in music generation and reviews various deep generative models and architectures that enable such control.

A. TYPES OF CONTROL FEATURES

Control features in symbolic music generation can be broadly categorized into:

- **Emotion and Mood**: Controlling the emotional content, such as happiness, sadness, tension [6].
- **Genre and Style**: Generating music in specific genres or styles, like classical, jazz, or pop [7, 8].
- **Key and Harmony**: Specifying key signatures, chord progressions, or harmonic structures [9, 10].
- **Rhythm and Tempo**: Adjusting rhythmic patterns, tempo, and rhythmic density [11].
- **Melody and Pitch**: Controlling melodic contours, pitch ranges, and note sequences [12].
- **Instrumentation and Texture**: Determining instrument combinations, track arrangements, and musical texture (monophonic, homophonic, polyphonic) [13, 14].
- **Structural Elements**: Shaping larger-scale structures like form, repetition, and development [15].

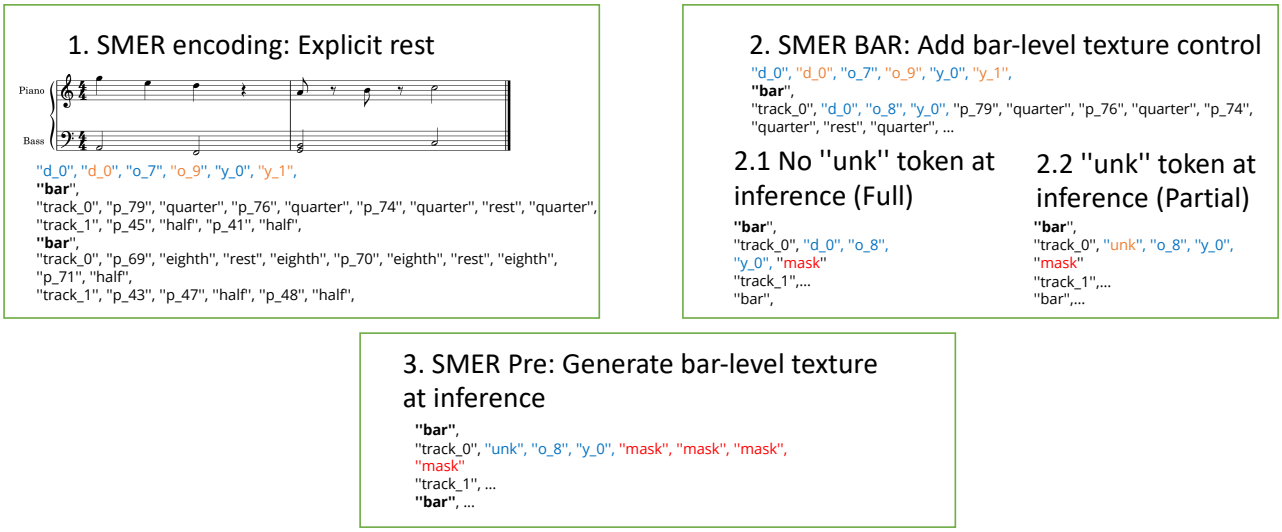

FIGURE 1. The workflow of this paper. 1. SMER encoding is proposed, which has an explicit rest token. 2. SMER-BAR: Texture control is added to the bar level. 2.1: At inference time, the "unk" token does not replace the bar-level textures. 2.2: At inference time, the "unk" token replaces bar-level textures. 3. SMER-Pre: The model generates bar-level textures after a bar's generation.

B. ARCHITECTURES FOR CONTROLLABLE MUSIC GENERATION

Various deep generative models have been applied to symbolic music to enable controllable generation. Below, we provide an overview of these architectures and how they incorporate control features.

1) Variational Autoencoders (VAEs)

VAEs facilitate controllable music generation by learning latent representations where control parameters can be encoded in specific dimensions. Users manipulate these latent dimensions to influence the generated music.

Brunner et al. [16] encodes control parameters into a subset of the latent space (z_{sub}), allowing manipulation of musical attributes. Models like those by Lim et al. [17], Guo et al. [18], and Tan and Herremans [19] provide users with the ability to fine-tune musical outputs through latent space adjustments.

2) Generative Adversarial Networks (GANs)

GANs, especially in style transfer applications, offer mechanisms for controlling musical style and genre. Brunner et al. [8] apply CycleGAN architectures to symbolic music, enabling the transfer of style between different musical pieces. Choi et al. [7] use Transformer encoders to predict a song's style, combined with a given melody, to generate music in the desired style.

3) Sequence Models (LSTMs and Transformers)

Sequence models are commonly used for conditional music generation, where conditioning is achieved by integrating control features into the model's inputs or embeddings.

- **Embedding Control Features**: Meade et al. [20] incorporate control embeddings directly into the token embeddings.
- **Emotion-Controlled Generation**: Sulun et al. [6] steer music generation by embedding valence and arousal information into a Transformer model, allowing control over the emotional content.

C. CONDITIONED TASKS AND CONTROL IN MULTI-TRACK MUSIC GENERATION

In multitrack music, specific tracks can be generated and conditioned on others:

- **Harmony Generation**: Producing harmony given a melody [10, 21].
- **Melody Generation**: Generating melodies based on chord progressions [9, 22].
- **Accompaniment Generation**: Using Transformer decoders to generate accompaniment tracks [14].

Depending on the task, the relevant control features can be explicitly added to the model's input to control specific music features. For instance, chord, bar position, and note durations generate note pitches in [12]. The VQ-VAE model by [11] offers bar-level control over several musical properties. Furthermore, [13] provides a web interface for setting track-level controls for density, polyphony, and occupation rate, which are then fed into a MMM model [5]. Recent advancements include a multi-track infilling model trained on a finely cleaned MIDI dataset [15]. This model shares the architecture and training strategy of [23] but does not include explicit control parameters. Conversely, [13] lacks fine-grained bar-level control, while [11] uses fixed bar-level control to represent global track-level control.

D. OUR APPROACH

This study aims to:

- **Investigating Hierarchical Control Interactions**: We explore how bar-level controls can influence track-level outcomes, enhancing overall controllability.
- **Developing Models with Integrated Controls**: By incorporating fine-grained bar-level control mechanisms into our models, we aim to give users precise control over musical attributes at multiple hierarchical levels, including bar and track levels.
- **Providing Practical Tools**: We make our models accessible through an Ableton Live plugin, facilitating interactive music composition and bridging the gap between AI models and real-world music creation.

III. METHODOLOGY

In this paper, we aim to improve the controllability of music infilling models through several musically meaningful parameters [5, 23]. In this section, we first introduce the proposed music infilling system and the control parameters. We then discuss the existing music encodings and the proposed ''SMER'' encoding, a symbolic music representation with explicit rest notation. Finally, we bring the idea of adding the bar-level texture controls to the SMER encoding alongside the existing controls.

A. CONTROLLABLE MUSIC INFILLING SYSTEM

In an infilling or inpainting task [5, 23, 24], the task of the model is to fill in the missing part given an input music context. Compared to the traditional left-to-right music continuation task [1], infilling allows for content generation at arbitrary positions within the music, making it valuable for interactive music applications where users can dynamically select regions for the system to fill in.

Our work adapts a vanilla Transformer encoder-decoder architecture [25] as the infilling model, based on Guo et al. [23]. The encoder input consists of corrupted sequences, where ''mask'' tokens replace certain parts of the input that need to be generated. The decoder input contains the same number of ''mask'' tokens as the encoder input, and the decoder's output targets are to reconstruct the masked tokens in the encoder input.

The basic generation unit is set to one track in one bar (e.g., the melody track in bar 3). This setup allows for control implementation at the bar level, with additional higher-level controls at the track level or song level. Control parameters are calculated or extracted from the original MIDI data, which, in our dataset, contains one to three tracks: melody, bass, and harmony.

The use of control tokens as prefixes (e.g., song-level key/tempo tokens at the sequence start) follows the success of ''prompt-based'' conditioning in models like CTRL [26], where control codes steer generation globally. Track-level tokens (e.g., d_5) act as intermediate constraints, similar to the ''coarse-to-fine'' control in VQ-VAE models [11].

Our model uses three levels of controls—**song-level**, **track-level**, and **bar-level**—to guide the infilling process:

- **Song-Level Controls:** Key (24 major/minor), tempo (7 bins), and time signature (4/4, 3/4, 2/4, 6/8).
- **Track-Level Controls:** Each track in a piece has a *density*, *polyphony*, and *occupation* rate, discretized into 10 categories (e.g., `d_0` to `d_9`). Additionally, an *instrument* token identifies the MIDI program.
- **Bar-Level Controls:** For each bar, we calculate the *tonal tension* (using [27]) and a finer-grained version of the same three texture controls (density, polyphony, occupation).

For complete details on how we compute these metrics and discretize them, please see Appendix A.

Music texture refers to the interaction between the different voices in the music [28]. It is vital in conveying musical ideas and relates to the organization of notes along the horizontal (time) and vertical (pitch) axes in standard music notation. Tonal tension is calculated based on each bar's spiral array theory [27, 29]. Continuous control features, such as note density, are discretized into different classes for modeling.

B. EXISTING SYMBOLIC MUSIC REPRESENTATIONS

Symbolic music representations are crucial for translating musical ideas into a format suitable for computational modeling. While several common symbolic music representations exist [30], the most prevalent types used in neural network-based music generation can be categorized into three groups: MIDI-like [31], event-based encodings such as REMI [4], and ABC notation [32].

However, these existing representations have limitations that can hinder the controllability and expressiveness of music generation models, such as implicit rest notation and too lengthy for polyphonic music. Those challenges can affect the model's ability to generate coherent and controllable music.

1) MIDI-like Formats

MIDI-like formats are based on the well-known ''Note On'' and ''Note Off'' MIDI events, representing when a note is sounded and silenced. In this representation, note durations are implicit and correspond to the time difference, known as ''delta time,'' between the ''Note On'' and ''Note Off'' events for a particular pitch. The delta time also includes any rest durations between notes.

For example, consider a melody track from the score shown in Figure 2, which includes three tracks over two bars. The MIDI-like representation of the melody track includes the sequence of events:

Figure 2 shows an example score with three tracks and two bars. The MIDI-like representation of the melody track of that score includes the sequence of events: ''NoteOn G5, 500, NoteOff G5, NoteOn E5, 500, NoteOff E5, NoteOn D5, 1000, NoteOff D5, NoteOn A4, 500, NoteOff A4, NoteOn B4, 500, NoteOff B4, NoteOn C5, 1000, NoteOff C5''. Here, the delta time is measured in milliseconds.

MIDI-like notation is flexible and has few restrictions, making it suitable for live recording and music staff notation. However, the freedom provided by MIDI-like notation has its

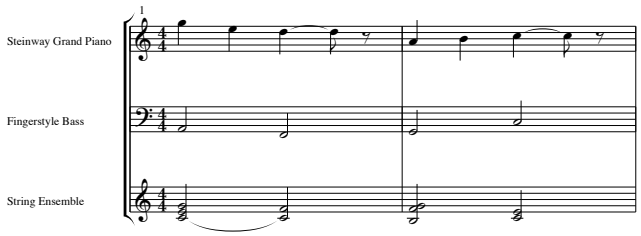

FIGURE 2. Music example with three tracks including melody, bass, and harmony.

downside. For instance, it can result in long-lasting notes if a ''note off'' event is not paired correctly with a ''note on'' event.

2) REMI Format

To address issues with unpaired ''Note On'' and ''Note Off'' events and implicit rests, Huang and Yang [4] proposed the REMI (REvamped MIDI-derived events) representation. REMI replaces time shift information with explicit ''Duration'' tokens, effectively discarding the ''Note Off'' events. It also introduces ''step'' tokens to mark the start positions of notes, with a fixed time step resolution (e.g., 16th notes).

For instance, the MIDI sequence ''NoteOn G5, 500, NoteOff G5'' can be represented in the REMI encoding as ''step 0, note G5, duration 500''. Rest tokens are implicit in the REMI encoding, so only the played notes are written out.

3) ABC Formats

ABC notation is primarily used to represent traditional folk music, typically monophonic. In ABC notation, a default note length is specified in the header using the ''L:'' tag. Each note's start time follows the end time of the previous note by default, and note durations can be modified using numerical modifiers.

For example, the first 14 notes of ''Twinkle, Twinkle, Little Star'' in ABC notation are represented as:

```
L:1/4 C C G G | A A G2 | F F E E | D D C2
```

In this example, ''L:1/4'' sets the default note length to a quarter note. The ''2'' after ''G'' and ''C'' indicates that the note duration is doubled (half note). Octave changes are handled using symbols like '','' (comma) to lower an octave or '''' (apostrophe) to raise an octave. However, the ABC format is not the most efficient if the music score has different melodies overlayed or separated in different voices as a mix of polyphonic notes.

C. SYMBOLIC MUSIC REPRESENTATION WITH EXPLICIT REST (SMER) FORMAT

Based on the limitations identified in the existing representations, we propose SMER (Symbolic Music representation with Explicit Rest notation). SMER introduces explicit rest tokens and aligns more closely with standard music notation, where the sum of the durations of notes and rests within each bar equals the bar duration defined by the time signature.

TABLE 1. The ''SMER'' coding scheme for symbolic music. The duration consists of five basic duration tokens, and other duration can be derived from combinations of the basic duration types. The ''rest, ''sep,'' and ''continue'' are also included in the duration category because they relate to note position and duration. The ''continue'' token, followed by the pitch token, can allow a note's duration to cross a bar boundary.

token type	token category	tokens	number
pitch	basic	p_21…p_108	88
duration	basic	whole, half, quarter, eighth, sixteenth, rest, sep, continue	8
structure	basic	bar, track_0, track_1,track_2	4
various	control	128 programs, 24 keys, 7 tempos, 4 time signatures, 30 track textures, 12 bar tensions	205
various	model	pad, eos, mask, unk	4
total			309

In SMER, note durations are represented using a combination of five basic duration types: whole, half, quarter, eighth, and sixteenth notes. The total duration of a note is the sum of these basic durations, similar to how durations are notated in standard music notation. Rests are explicitly represented using rest tokens with corresponding durations.

By explicitly representing rests and aligning with standard notation, SMER provides a more musically meaningful encoding. This explicitness can help the model better understand musical timing, phrasing, and structure, potentially enhancing controllability in music generation tasks.

The vocabulary of the SMER coding scheme is presented in Table 1:

In SMER, we use five basic duration tokens, including ''whole'', ''half'', ''quarter'', ''eighth'', ''sixteenth'' notes. By default, the start time of a note or rest follows the end time of the previous note. To represent several notes starting at the same time, or having partial overlaps between their duration, a special ''sep'' token is added here to allow us to change the start time of a note to the same start time as the previous note. In addition, ''rest'' tokens can be used to set the next token's start time to the right place further down in time. Like notes, ''rest'' tokens start after the previous note ends.

To represent a pitch longer than a ''whole'' note, a ''continue'' token is added at the start of the next bar to elongate the duration of a note in the previous bar. If a note ''p_67'' lasts longer than a whole note, a ''continue'' will be put in the next bar, followed by the same pitch declaration

''p_67 whole''.

There can be several tracks in a bar, and each track starts with a ''track_num'' token to indicate its sequence number. In this research, the input music contains up to three tracks: melody, bass, and harmony. The melody is marked as ''track_0'', the bass as ''track_1'', and the harmony as ''track_2''.

The control tokens are inserted into the original SMER encodings at different positions. The song control tokens for time signature, tempo and key are inserted at the start, and the track-level texture control tokens follow the song control tokens but occur before the first ''bar'' token. The bar tonal tension control tokens are inserted after each ''bar'' token. An example with song-level, track-level texture, and bar-level tension control for the music in Figure 2 is shown below. For this example, the control tokens are coloured differently for each track they control. Please refer to III-A for each control token's meaning.

- **Orange**: Song-level control tokens (time signature, tempo, key)
- **Cyan**: Control tokens for melody track (''track_0'')
- **Brown**: Control tokens for bass track (''track_1'')
- **Red**: Control tokens for harmony track (''track_2'')
- **Teal**: Bar-level tonal tension control tokens

4/4, t_3, k_0,
d_0, d_0, d_0, o_9 o_9, o_9, y_0, y_0, y_9, i_0, i_0, i_0,
bar, s_2,
track_0, p_79, quarter, p_76, quarter, p_74, quarter, eighth,
rest, eighth,
track_1, p_45, half, p_41, half,
track_2, p_64, p_67, half, sep, p_60, whole, sep, half, p_65,
half,
bar, s_1,
track_0, p_69, quarter, p_71, quarter, p_72, quarter, eighth,
rest, eighth,
track_1, p_43, half, p_48, half,
track_2, p_59, p_65, p_67, half, p_60, p_64, half

Given that we focus on the infilling task in this paper, we will be masking part of the input so that our model can generate these missing parts. If the first bar of melody and the second bar of harmony track of the previous input are masked, the transformer encoder input is:

4/4, t_3, k_0,
d_0, d_0, d_0, o_9 o_9, o_9, y_0, y_0, y_9, i_0, i_0, i_0,
bar, s_2,
mask,
track_1, p_45, half, p_41, half,
track_2, p_64, p_67, half, sep, p_60, whole, sep, half, p_65,
half,
bar, s_1,
track_0, p_69, quarter, p_71, quarter, p_72, quarter, eighth,
rest, eighth,
track_1, p_43, half, p_48, half,
mask

The decoder input is: **mask**, **mask**
The decoder target output is:

track_0, p_79, quarter, p_76, quarter, p_74, quarter, eighth,
rest, eighth,
track_2, p_59, p_65, p_67, half, p_60, p_64, half

D. ADDING BAR-LEVEL TEXTURE TO SMER

We introduced different control types in Section III-A. However, these track-level texture controls have limitations when specifying textures at the bar level. For instance, if a user wishes to increase the number of notes in the second bar of the bass track, the track-level density control alone does not provide the necessary granularity. In such cases, users may need to iteratively generate and adjust the music until achieving the desired result, which is inefficient and time-consuming.

We propose incorporating bar-level texture controls into the SMER encoding to enhance controllability and allow for more precise adjustments. We explore three approaches to investigate the interactions between bar-level and track-level control tokens. These approaches correspond to the points 2 and 3 in Figure 1:

1) Full Condition This approach aligns the bar-level texture controls with the track-level texture controls by setting the bar-level textures to values similar to the track-level textures. For example, if the track-level density of the melody track is set to a high level, the bar-level density for each bar of the melody track is also set to a high level. This simultaneous adjustment ensures that the local targets (density in each bar) and the global target (density across all bars) correlate positively, reinforcing the overall track-level control.
2) Partial Condition This approach introduces a ''free state'' for specific bar-level textures by replacing them with an ''unknown'' token (''unk''). When a track-level texture parameter changes, the corresponding bar-level texture parameters for that track are set to ''unk,'' allowing the model to decide the bar-level textures during generation freely. The other bar-level textures remain unchanged. For instance, if the melody track's track-level density is altered, the melody track's bar-level density tokens are replaced with ''unk.'' In contrast, the occupation and polyphony bar-level textures remain the same. This approach allows the model to generate bar-level textures while maintaining some constraints.
3) Prediction Condition This approach involves the model predicting the bar-level textures after generating each bar, leveraging the autoregressive nature of sequence models. During training, the model is taught to generate bar-level texture tokens alongside musical content. At inference time, the model generates the bar-level textures, which are then fed back into the model to guide subsequent generations. By providing explicit bar-level texture information during generation, the

model may adjust future outputs to better align with the desired track-level texture controls.

In both the "Partial Condition" and "Prediction Condition," we introduce an "unk" token to represent the "unknown" or "free" state for bar-level textures in the input. When a bar-level texture is replaced with "unk," it signals to the model that no specific condition is provided, and the model should determine the texture autonomously. Different strategies are applied during training to expose the model to the "unk" token, which we discuss in detail in Section IV-B. At inference time, the corresponding bar-level textures are replaced with the "unk" token based on the chosen approach.

Below, we provide examples illustrating the "Full Condition," "Partial Condition," and "Prediction Condition." These examples are based on the music in Figure 2. The texture control tokens are colour-coded for clarity, and non-essential tokens are omitted for brevity. In these examples, the melody track's track-level density is changed from level 0 to level 5, and the bar-level density of the melody track is adjusted accordingly in each condition. The altered track-level and bar-level tokens are highlighted in *italic* font.

1) Full condition

In the "Full Condition," the bar-level textures of the target track are set to values similar to the track-level texture controls, reinforcing the track-level control at the bar level.

d_5,… o_9,… y_0, …
bar, s_2, track_0,*d_5*,o_9,y_0,**mask**,
track_1,…,track_2,
bar, s_1, track_0,*d_6*,o_9,y_0, **mask**,
track_1, …,track_2,…,
In this example: The melody track's track-level density is set to *d_5*.The bar-level density for the melody track in both bars is set to values similar to the track-level density (*d_5* and *d_6*).The "mask" tokens indicate the parts to be generated by the model.

2) Partial condition

In the "Partial Condition," the bar-level textures corresponding to the changed track-level texture are replaced with the "unk" token, giving the model flexibility in determining these bar-level textures. Other bar-level textures remain unchanged.

d_5,… o_9,… y_0, …
bar, s_2, track_0,*unk*,o_9,y_0,**mask**,
track_1,…,track_2,
bar, s_1, track_0,*unk*,o_9, y_0,**mask**,
track_1, …,track_2,…,
In this example: The melody track's track-level density is set to *d_5*. The bar-level density for the melody track is replaced with *unk*, allowing the model to generate the bar-level density freely. The occupation (o_9) and other bar-level textures remain unchanged. The "mask" tokens indicate the parts to be generated by the model.

3) Prediction condition

In the "Prediction Condition," during training, the model learns to predict the bar-level textures after generating each bar. The bar-level texture tokens are copied to the end of the bar sequence and replaced with "mask" tokens. At inference time, the model generates these bar-level textures, which are then used to guide subsequent generations.
The bar-level texture prediction "Prediction Condition" draws inspiration from hierarchical planning in text generation [33], where predicting high-level onlines (e.g., story structure) before generating text improves coherence. Similarly, our model predicts bar-level textures autoregressively, enabling dynamic alignment between local (bar-level) and global (track-level) constraints.

d_5,… o_9,… y_0, …
bar, s_2, track_0,*unk*,o_9,y_0,
mask,**mask**, **mask**,**mask**,
track_1,…,track_2,
bar, s_1, track_0,*unk*,o_9, y_0,
mask,**mask**,**mask**,**mask**,
track_1, …,track_2,…,
In this example: The melody track's track-level density is set to *d_5*. The bar-level density for the melody track is replaced with *unk*. After each bar, additional "mask" tokens are added to represent the bar-level textures to be predicted. During inference, the model generates both the musical content and the bar-level textures, which are then fed back into the model to guide the generation of subsequent bars.

By comparing these three approaches, we aim to understand how bar-level texture controls influence track-level texture outcomes and overall controllability.

IV. IMPLEMENTATION

A. DATA PRE-PROCESSING

The Lakh dataset consists of 178,561 MIDI files collected from the internet. However, a significant portion of these files are of low quality. Therefore, carefully selecting files ensures that the model learns valuable music knowledge. Using the MIDI-Miner Toolkit [34], 173,974 files with at least one melody, bass, or harmony track were identified. One was randomly chosen as the harmony track in cases where both a chord and an accompaniment track existed. The selected files meet the criteria of having a time signature of "4/4," "3/4," "6/8," or "2/4," with no time signature or tempo changes throughout the song. After filtering the file time signature, length and tempo, 125,589 songs remain for SMER and REMI encodings.

After encoding the dataset, the control tokens are calculated and added to enable an experiment that compares

SMER and REMI. Each bar's tonal tension (tensile strain) and song key are calculated using the MIDI-Miner [34]. Some songs' keys could not be derived using MIDI-Miner and were thus discarded, reducing the total dataset to 93,845 files.

The dataset is split into training, validation, and test sets with a ratio of 0.8, 0.1, and 0.1, respectively. The SMER and REMI datasets have the same files to ensure a fair comparison, and the test set contains 9,385 files. Among these test files, 3,616 contain all three tracks (melody, bass, and harmony), and those files were selected for controllability comparison. This selection allows for a comparison of the controllability of each track, given the other two tracks as conditions.

Each entry in the dataset contains a maximum of 16 bars, which are derived by using a moving window with a step of 8 bars on each song. The largest sequence length for one entry is 2,200 in the dataset for SMER/REMI, and that length limit is to restrict the total token length for GPU usage. Track control tokens, including track density, polyphony, and occupation rate, are calculated for each track in each dataset entry, considering a maximum of 16 bars.

B. MODEL ARCHITECTURE AND TRAINING

We adopt the same model architecture as Guo et al. [23], utilizing a Transformer encoder-decoder for the music infilling task. The encoder's input consists of masked tokens, and the decoder's target output is the sequence of tokens to be infilled. The encoder and decoder have four layers and eight attention heads, with a model dimension 512.

One "mask" token represents a track in a bar (e.g. melody track in bar 3), and the decoder's target is reconstructing that masked bar track. Each "mask" in the encoder input is matched with a "mask" input in the decoder, and the decoder target output will end with an "eos" token. A "pad" token is also added to pad sequences of different lengths to match the maximum sequence length in this batch.

The training process includes two stages: pretraining and fine-tuning.

1) Pretraining Stage

The pretraining stage allows the model to learn basic musical grammar. During pretraining, random spans of tokens with lengths between 1 and 3 are masked, and the decoder is trained to reconstruct the original tokens within these spans. The spans of lengths 3, 1, and 2 occur with a frequency ratio of 2:1:1, respectively. These choices are derived from an ablation study. In total, 15% of the tokens in the input are masked during pretraining.

2) Fine-Tuning Stage

The fine-tuning stage trains the model to generate masked tracks in masked bars, simulating the infilling task. Three different masking strategies are applied:

1) Random Bars in Random Tracks (40% occurrence): Random tracks within random bars are masked.
2) Whole Tracks (30% occurrence): One or several entire tracks are masked.
3) Whole Bars (30% occurrence): One or several entire bars are masked.

In the SMER models with added bar-level texture controls (SMER Bar and SMER Pre), the "unk" (unknown) token is used differently during training to represent missing bar-level textures:

1) Pretraining Stage: 5% of the control tokens are randomly replaced with "unk" tokens.
2) Fine-Tuning Stage:
    a) When masking random bars in random tracks or whole bars (all the tracks in that bar):
        i) 10% chance of replacing all three textures with "unk".
        ii) 10% chance for a random combination of two textures.
        iii) 10% chance for one random texture.
        iv) 70% chance for no corruption.
    b) When masking whole tracks:
        i) 10% chance of replacing all three textures with "unk".
        ii) 25% chance for a random combination of two textures.
        iii) 40% chance for one random texture.
        iv) 25% chance for no corruption.

This training strategy exposes the model to missing bar-level textures, teaching it to handle the "unk" token appropriately. During inference, different bar-level textures can be replaced with "unk" to implement the "partial" and "prediction" conditions described earlier.

V. EVALUATION

In this section, we aim to evaluate the controllability and performance of our proposed models in the music infilling task. Our primary research questions are:

1) Comparison of Encodings: How does the proposed SMER encoding compare with the widely used REMI encoding regarding model accuracy and controllability?
2) Effect of Bar-Level Texture Controls: How does incorporating bar-level texture controls influence track-level texture controllability, and which bar-level structure (full, partial, prediction) allows for the best control?

To address these questions, we conduct experiments using four models:

1) REMI: A baseline model using the REMI encoding without bar-level texture controls.
2) SMER: A model using our proposed SMER encoding without bar-level texture controls (corresponding to Figure 1, point 1).
3) SMER Bar: A model using the SMER encoding with bar-level texture controls added to the input

(corresponding to the ''full'' and ''partial'' condition in Figure 1, point 2). The difference between the ''full'' and ''partial'' conditions is applied during inference.

4) SMER Pre: A model using the SMER encoding with bar-level texture controls both added to the input and predicted in the output (corresponding to the ''prediction'' condition in Figure 1, point 3).

We evaluate the four models' performance and controllability using quantitative metrics and qualitative analysis. Our evaluation focuses on:

1) Model Metrics Comparison: Comparing the models' training metrics, including the accuracy of their outputs for different token types.
2) Controllability Analysis: Assessing how different encoding schemes and control token conditions affect the models' controllability.

A. MODEL METRICS COMPARISON

We compare the four models based on their accuracy in predicting various token types at the last training epoch. All models include key, tensile strain (tonal tension), and three types of texture controls (density, polyphony, occupation rate) in their inputs.

The total accuracy is the average accuracy of all output tokens. In ''SMER Pre,'' the total accuracy also includes the accuracy of the predicted bar-level texture tokens. The pitch accuracy refers to the accuracy for pitch tokens. The duration accuracy for the three SMER models pertains to the duration tokens listed in Table 1. For the REMI model, duration tokens include the ''step'' and ''duration'' tokens, which are needed to determine a note's timing.

Our results in Table 2 show:

1) Total Accuracy: The SMER and REMI models have similar total accuracy. After adding bar-level textures, the ''SMER Bar'' model achieves better total and duration accuracy. The ''SMER Pre'' model has a higher total accuracy due to including highly accurate predicted bar-level texture tokens.
2) Pitch and Duration Accuracy: ''SMER Bar'' and ''SMER Pre'' have similar pitch and duration accuracy, both slightly better than ''SMER'' and ''REMI''.

TABLE 2. The last epoch's accuracy of different models. The total accuracy is the average accuracy of all the output tokens. The pitch accuracy is the accuracy for pitch tokens, and the duration accuracy for the three SMER models is for the duration tokens listed in Table 1. The REMI duration tokens include the "step" and "duration" in the REMI format because those two types of tokens together decide a note's time. "SMER" and "REMI" do not have bar-level texture tokens. The "SMER Bar" includes the bar-level texture in the input, and "SMER Pre" is the model with bar-level texture prediction. "SMER" and "REMI" has similar total accuracy, and after adding bar-level textures in the input, the models have better duration and total accuracy. It is noted that "SMER Pre" also output bar-level texture tokens in the output, which is included in its total accuracy.

	SMER	REMI	SMER Bar	SMER Pre
Pitch	0.768	0.789	0.772	0.771
Duration	0.856	0.846	0.884	0.884
bar-level density	NA	NA	NA	0.996
bar-level occupation	NA	NA	NA	0.973
bar-level polyphony	NA	NA	NA	0.987
total	0.835	0.832	0.852	0.893

B. CONTROLLABILITY ANALYSIS

We assess the models' controllability to understand the effects of different encoding schemes and control token conditions. Specifically, we compare the controllability between models trained with SMER and REMI encodings and evaluate how incorporating bar-level texture controls influences the models' ability to adhere to specified controls.

1) Controllability Comparison Between SMER and REMI

We compare five pairs of models, each pair consisting of a SMER-based model and a REMI-based model trained with the same controls. The first four pairs are trained with one type of control added (either bar tension or one of the three track texture controls). The last pair includes models trained with all controls, corresponding to the ''SMER'' and ''REMI'' models in Table 2.

To test controllability:

1) Bar Tension Control: We select random bars and change their tensile strain to random levels. The models generate music with these modified tension levels. We compare the generated bar tensions with the set tension levels. A lower average difference indicates higher controllability.
2) Track Texture Control: We select a random track and set one of its texture parameters (density, polyphony, or occupation rate) to a random level. The models generate the entire track, and we compare the generated track's texture with the set control. A lower average difference indicates higher controllability.

When we compare models trained with one type of control, the key findings in Table 3 include:

1) SMER Advantages:
   a) Better controllability in bass occupation ($p = 0.02$) and harmony occupation ($p < 0.001$).
   b) Better controllability in bass polyphony ($p = 0.003$). This may be due to SMER's explicit ''rest'' tokens, making it easier for the model to calculate note durations and rests.
2) REMI Advantages:
   a) Better controllability in melody density ($p = 0.044$).
   b) Better overall density control ($p = 0.07$).

Since note density is directly related to pitch tokens, and both SMER and REMI have the same number of pitch tokens,

REMI's simpler representation may aid in density control.

When all controls are added, controllability generally decreases for both models as in Table 4. Significant differences include:

1) Bar Tonal Tension: SMER performs better ($p = 0.003$).
2) Bass Polyphony: REMI performs better ($p < 0.001$).

These differences may result from model complexity constraints or trade-offs when controlling multiple variables simultaneously.

In our experiments, we observe that SMER does not consistently outperform REMI under all multi-control scenarios, but it demonstrates clear advantages in specific cases. When handling bass and harmony occupation rate, the explicit rest tokens in SMER appear to help the model represent and predict silent durations more accurately, leading to lower average deviations between the desired and generated values. This aligns with our hypothesis that making rests explicit makes it easier for the model to encode and decode longer or more frequent periods of silence—an important factor in bass and harmony parts. In contrast, REMI generates stronger results for melody density and bass polyphony when multiple controls are applied simultaneously. Overall, the results suggest that the explicit rests in SMER enhance control of durations and silences in certain tracks, whereas REMI occasionally retains an advantage for dense melodic lines and polyphonic settings, illustrating the inherent trade-offs when controlling multiple musical aspects simultaneously

These findings suggest that the choice of encoding alone may not be sufficient to enhance controllability in complex models with numerous control parameters. The benefits of SMER's musically meaningful representation might be offset by the increased complexity introduced when integrating multiple controls. Therefore, while SMER offers some advantages in specific aspects, further research is needed to fully leverage its potential.

2) Trade-off Analysis Between Occupation and Polyphony

To investigate whether changes in one control (e.g., occupation) inadvertently alter another control (polyphony), we analyzed the difference values (Δo for occupation and Δy for polyphony) in two scenarios: (1) when polyphony was explicitly controlled, and (2) when occupation was explicitly controlled. We then computed the Pearson correlation between these difference values for each scenario. The harmany track is selected for this task because of its diverse polyphony levels compared to melody or bass. The SMER model with all controls is selected for this test.

- **Polyphony Control Scenario.** We found no significant linear relationship between Δo and Δy ($r \approx 0.069$).
- **Occupation Control Scenario.** The correlation between Δo and Δy was again near zero ($r \approx -0.005$).

These results suggest that adjusting one parameter (occupation or polyphony) does not systematically push or pull the other. In other words, there is effectively *no trade-off* between occupation and polyphony for the harmony track under the tested conditions.

3) Fine-Grained Control with Bar-Level Texture

We evaluate track-level texture controllability in the SMER models under the ''full,'' ''partial,'' and ''prediction'' bar-level conditions. The results are presented from Table 5 to Table 8, with ''original'' referring to the ''SMER'' model without bar-level textures in Table 2.

1) Full Condition: Adding bar-level textures significantly improves track-level texture controllability when each bar's texture is set similar to the track-level texture.
2) Partial Condition: When one bar-level texture is replaced with ''unk,'' giving the model freedom for that texture, the controllability for that texture decreases compared to the ''original'' model. However, controllability for the other two textures improves due to the additional bar-level constraints.
3) Prediction Condition: The model predicts the missing bar-level textures during generation, which helps adjust subsequent bars to align with the desired track-level textures. This condition often performs better or is similar to the partial condition.

In Table 5, we compare the track-level texture controllability of the SMER model under the ''full,'' "partial," and "prediction" bar-level conditions. The "original" column refers to the SMER model without bar-level textures, as presented in Table 2. Adding local bar-level textures significantly improves the controllability of the track-level textures when each bar's texture is set to a value similar to the track-level texture, as seen in the full condition.

In the partial condition, where freedom is given to one of the bar-level textures by replacing it with the "unk" token, the track-level controllability for that specific texture is worse than in the original model. This decrease is reasonable because the model has fewer constraints for the unconstrained texture, leading to greater variability in the generated output. However, when the bar-level density is replaced by "unk," the remaining bar-level textures—polyphony and occupation—are still constrained and may help improve their corresponding track-level textures.

Adding local bar-level textures significantly improves the controllability of the track-level textures when each bar's texture is set to a value similar to the track-level texture, as seen in the full condition.

In the partial condition, where freedom is given to one of the bar-level textures by replacing it with the ''unk'' token, the track-level controllability for that specific texture is worse than in the ''original'' model. This decrease is reasonable because the model has fewer constraints for the unconstrained texture, leading to greater variability in the generated output. However, when the bar-level density is replaced by ''unk,'' the remaining bar-level textures—polyphony and occupation—are still constrained and may help improve their corresponding track-level textures.

TABLE 3. The controllability comparison between the model with only one of the bar tension and track-level texture controls in SMER and REMI encodings. An independent t-test is implemented for each generated/set music control. Each control is trained on a separate model. "df" is the degree of freedom of the t-test. The p values are corrected by the Holm-Bonferroni method.

control name	track	SMER mean	REMI mean	mean diff	df	corrected p value
tensile strain		1.84	1.83	-0.01	38734	1
density	melody	0.97	0.87	**-0.1**	2390	**0.044**
	bass	1.06	0.94	**-0.11**	2369	**0.072**
	harmony	0.86	0.89	0.03	2403	1
occupation	melody	0.70	0.66	-0.04	2365	0.666
	bass	0.75	0.86	**0.11**	2395	**0.02**
	harmony	0.80	1.01	**0.22**	2315	**<.001**
polyphony	melody	2.12	2.14	0.01	2455	1
	bass	3.04	3.43	**0.39**	2319	**0.003**
	harmony	1.78	1.86	0.08	2415	1

TABLE 4. The controllability comparison between the model trained with SMER and REMI encodings all controls. "df" is the degree of freedom of the t-test. The p values are corrected by Holm-Bonferroni method.

control name	track	SMER mean	REMI mean	mean diff	df	corrected p value
tensile strain		1.80	1.86	0.06	38824	**0.003**
density	melody	1.21	1.11	-0.09	2366	0.66
	bass	1.3	1.32	0.02	2374	1
	harmony	1.2	1.24	0.03	2450	1
occupation	melody	0.90	0.94	0.04	2375	1
	bass	1.03	1.04	0.01	2420	1
	harmony	1.4	1.34	-0.06	2357	1
polyphony	melody	2.03	2.07	0.14	2415	0.58
	bass	3.24	2.5	-0.75	2410	**<.001**
	harmony	2.01	2.08	0.14	2370	0.66

Table 6 verifies that in the partial and prediction conditions, the model exhibits better occupation and polyphony track-level texture control than the full and original conditions when the bar-level density is unconstrained. This suggests that even though one texture is left free, the additional constraints from the other two bar-level textures can enhance the controllability of those textures at the track level. This conclusion holds when the target track texture control is polyphony in Table 7 or occupation in Table 8. In these cases, replacing the bar-level polyphony or occupation with "unk" in the partial condition leads to a decrease in controllability for the unconstrained texture but an improvement in controllability for the other two textures. This indicates a compensatory effect where constraining certain textures at the bar level can bolster their controllability at the track level, even when other textures are left unconstrained.

Furthermore, in the prediction condition, where the model predicts the missing bar-level textures during generation, the track-level texture controllability often performs better or matches that of the partial condition. This improvement demonstrates that predicting bar-level textures helps the model adjust subsequent bar generations to align more closely with the desired track-level texture controls.

In summary, while adding bar-level textures significantly enhances track-level texture controllability in the full condition, the partial condition shows that unconstrained bar-level textures can negatively impact controllability for that texture but improve it for the constrained textures. The prediction condition offers a balance by allowing the model to compensate for the unconstrained textures through prediction, resulting in overall better controllability.

TABLE 5. Track-level texture controllability under different bar texture conditions. "original" is the input without local bar-level texture. "full" is the input with all the bar-level textures. "partial" is the input with one of the bar-level texture missing. "prediction" is same as "partial" but predict the missing texture in the process of generation.

texture change	track	track-level texture controllability			
		original	bar texture conditions		
			full	partial	prediction
density	melody	1.21	0.53	1.76	1.39
	bass	1.3	0.39	1.45	1.06
	harmony	1.2	0.62	1.42	1.36
occupation	melody	0.90	0.56	1.52	1.41
	bass	1.03	0.52	1.15	1.15
	harmony	1.4	0.75	2	1.83
polyphony	melody	2.03	0.77	2.12	1.73
	bass	3.24	1.3	4.03	2.99
	harmony	2.01	1	2.11	2.1

We also analyze the bar-level texture controllability, and the result is in Table 9. When comparing the generated bar-level textures with the set or predicted textures, we find the predicted bar-level textures closely match the actual

TABLE 6. The occupation and polyphony track-level controllability when the track density is set to a different value. The "partial" and "prediction" conditions are better controllable for the unchanged track texture. "ori": original.

track name	occupation and polyphony control							
	occupation				polyphony			
	ori	full	partial	prediction	ori	full	partial	prediction
melody	0.94	0.82	0.26	0.27	0.56	0.07	0.06	0.04
bass	0.83	0.54	0.28	0.26	0.20	0.03	0.03	0.03
harmony	0.89	0.57	0.28	0.24	0.78	0.46	0.32	0.30

TABLE 7. The density and occupation track-level controllability when the track density is set to a different value. The "partial" and "prediction" conditions are better controllable for the unchanged track texture.

track name	density and occupation control							
	density				occupation			
	ori	full	partial	prediction	ori	full	partial	prediction
melody	0.65	0.14	0.12	0.12	0.70	0.27	0.24	0.25
bass	0.67	0.13	0.11	0.1	0.76	0.26	0.21	0.2
harmony	0.75	0.21	0.15	0.14	0.75	0.25	0.21	0.21

generated textures, especially for bar-level density. This confirms that predicting bar-level textures effectively guides the model's generation process.

In conclusion:

1) Bar-Level Textures Enhance Control: Incorporating bar-level textures significantly improves track-level texture controllability in the full condition.
2) Prediction Aids Adjustment: Predicting missing bar-level textures (prediction condition) helps the model adjust subsequent generations to better align with the desired track-level textures.
3) Trade-Offs in Partial Condition: The partial condition offers flexibility but may reduce controllability for the unconstrained texture.

C. COMPARISON WITH MMM

MMM [5] is another representative music infilling model, trained on various genres and supports more than 3 tracks and more types of note durations. The model ''SMER Pre'' is compared with MMM V2, by using the song *Imagine* by John Lennon, and that file is in the Supplementary material.

1) Music Quality Evaluation

To evaluate the quality of the generated music quantitatively, we experimented using the melody track of the song *Imagine* by John Lennon. We generated 100 versions of the melody track using both our SMER Pre model and the Multi-Track Music Machine (MMM) V2 model for comparison. We then analyzed the distributions of pitch, duration, and the average number of notes in the generated melodies to assess how closely they resemble the original. In this experiment, all original control parameters of the music were kept unchanged to focus solely on the models' ability to reproduce the melody with high fidelity.

Figure 3 shows the pitch distribution of the generated melodies. Compared to the original song in C major, the MMM-generated melodies exhibit a wider range of pitches, including more notes that do not belong to the C major scale. This increased variability may be due to MMM's diverse training corpus, which includes a broad range of genres beyond pop music. In contrast, the SMER-generated melodies maintain a pitch distribution closely aligned with the original song, preserving the key signature and harmonic content.

Figure 4 illustrates the duration distribution of the generated notes. The MMM model outputs a higher frequency of sixteenth notes, resulting in faster and more intricate rhythmic patterns. The SMER Pre model's generated melodies, however, display a duration distribution more similar to the original melody.

Figure 5 compares the average number of notes in the generated melodies. The original melody contains 58 notes. The SMER Pre model's generated melodies have an average note count closer to this number, while MMM's generated melodies deviate more significantly.

These results indicate that our SMER Pre model produces music that is more faithful to the original song in terms of pitch, rhythm, and structural elements, demonstrating superior quality compared to MMM V2.

TABLE 8. The density and polyphony track-level controllability when the track occupation is set to a different value. The "partial" and "prediction" conditions are better controllable for the unchanged track texture.

track name	density and polyphony control							
	density				polyphony			
	ori	full	partial	prediction	ori	full	partial	prediction
melody	0.79	0.18	0.15	0.13	0.35	0.08	0.08	0.05
bass	0.71	0.15	0.11	0.11	0.14	0.03	0.03	0.02
harmony	0.93	0.22	0.17	0.17	1.11	0.89	0.42	0.37

2) Controllability comparison

We further evaluated the controllability of our SMER Pre model by comparing it with the MMM V2 model. MMM offers track-level density control with ten different levels, minimum and maximum note duration controls and six levels of polyphony rate controls. Since both models provide density control with comparable levels, we selected track-level density as the parameter for comparison.

Using *Imagine* as the starting point, we randomly altered the track-level density to new levels for each of the three tracks (melody, bass, and harmony). For each track, we generated 100 versions, each time applying a different random density level. We then compared the generated track and set densities to evaluate how accurately each model adheres to the specified control parameters. The generation process was repeated 100 times for each track to ensure statistical reliability. The files used in this experiment are provided in the supplementary materials.

Table 10 presents the results of the track density controllability comparison. The table shows the average difference between the set density levels and the actual

TABLE 9. The bar-level texture controllability. Column "track" shows which track-level texture is changed. Row "bar" shows the bar-level texture controllability when that track-level texture changes. The corresponding bar-level texture is "unked" when the track-level texture changes in "partial" and "prediction" conditions. The cells not in bold font show the average difference between the set bar-level texture and the generated bar-level texture. The cells in bold font show the average difference between the predicted bar-level texture and the generated bar-level texture.

bar → / track ↓	track name	density			occupation			polyphony		
		full	partial	prediction	full	partial	prediction	full	partial	prediction
density	melody	0.67	NA	**0.11**	0.18	0.26	0.2	0.14	0.23	0.19
	bass	0.54	NA	**0.1**	0.14	0.15	0.13	0.13	0.12	0.13
	harmony	0.84	NA	**0.14**	0.23	0.30	0.18	0.21	0.24	0.16
occupation	melody	0.82	0.61	0.54	0.79	NA	**0.34**	0.27	0.52	0.47
	bass	0.54	0.46	0.45	0.80	NA	**0.37**	0.26	0.42	0.42
	harmony	0.57	0.53	0.42	0.97	NA	**0.36**	0.25	0.43	0.34
polyphony	melody	0.07	0.13	0.11	0.06	0.13	0.1	1.87	NA	**0.49**
	bass	0.03	0.08	0.08	0.04	0.07	0.07	1.86	NA	**0.35**
	harmony	0.46	0.6	0.56	0.94	0.67	0.61	1.86	NA	**0.51**

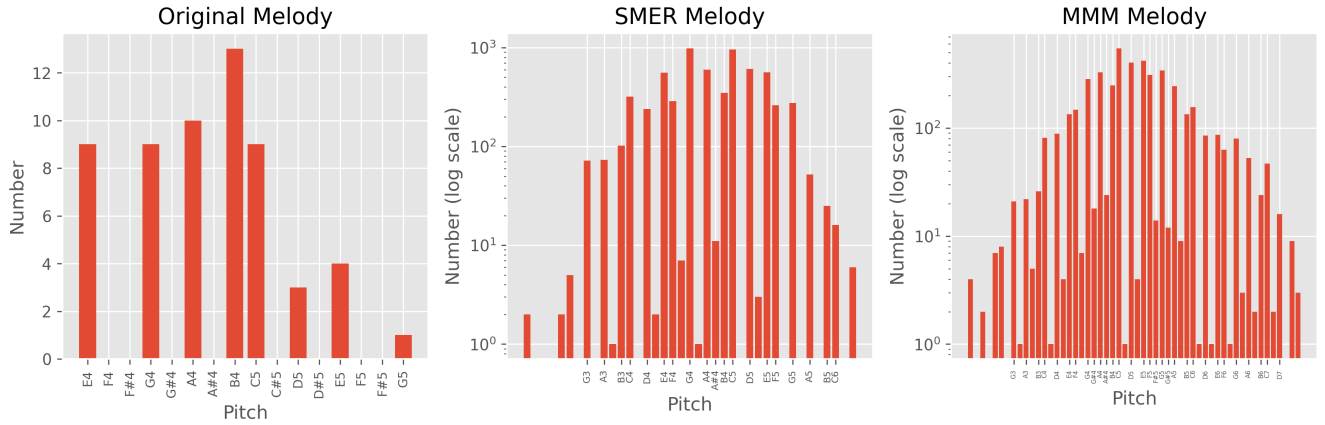

FIGURE 3. The melody pitch distribution comparison between original, SMER and MMM generated result.

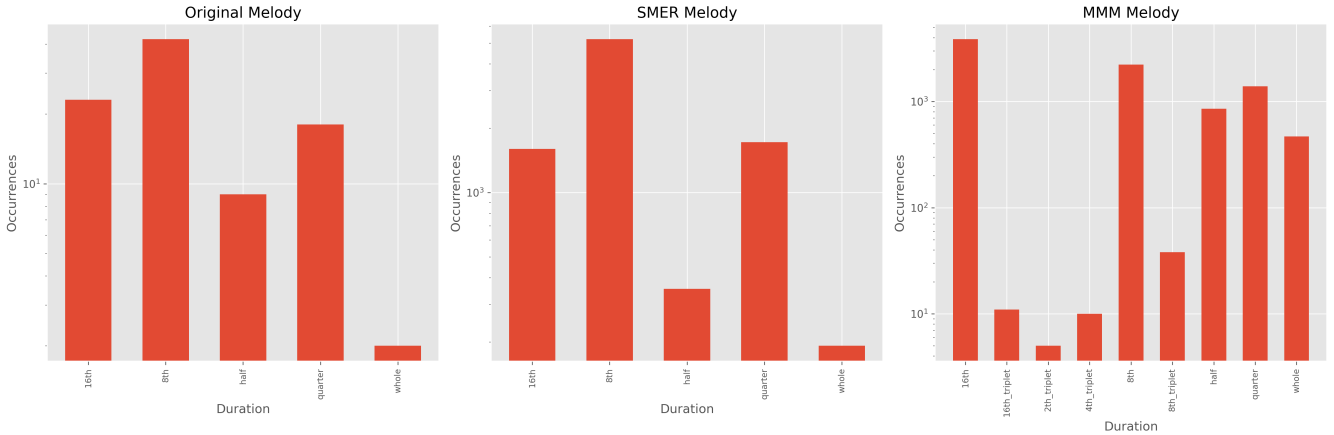

FIGURE 4. The melody duration distribution comparison between original, SMER and MMM generated result.

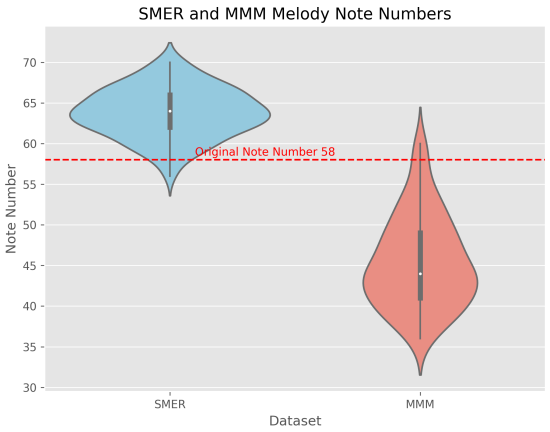

FIGURE 5. The SMER and MMM generated melody note number distribution.

TABLE 10. The controllability comparison between SMER and MMM. The *Imagine* was selected as the template song with melody, bass and harmony tracks. Each track's density is set to a random value, and that track is generated 100 times. The average difference between the set/generated density is calculated as controllability. SMER has much better controllability in all three tracks.

track	controllability	
	SMER	MMM
melody	1.07	2.62
bass	0.64	2.85
harmony	1,42	3.58

densities achieved in the generated music for each model and track.

Our results demonstrate that the SMER Pre model exhibits significantly better controllability in all three tracks than MMM V2. Specifically, the SMER Pre model's generated tracks closely match the specified density levels, with minimal deviation. In contrast, MMM V2 shows larger discrepancies between the set and achieved densities. This indicates that our model provides more precise control over the musical texture at the track level, allowing users to achieve their desired creative outcomes more effectively.

D. USER SURVEY

To further evaluate the quality of the generated music and compare our SMER model with the MMM system, we conducted a user survey consisting of three sections. A total of 24 participants completed the survey. Most participants were music enthusiasts or amateurs, and their familiarity with AI was evenly distributed among "Yes," "No," and "Somewhat." The user profiles are summarized in Table 11. To ensure consistency in audio quality, the generated symbolic music was rendered using selected virtual instruments and audio effects in Ableton Live. Each audio file was synthesized using the same settings for a fair comparison.

The first part of the survey aimed to assess whether listeners could distinguish between AI-generated music and

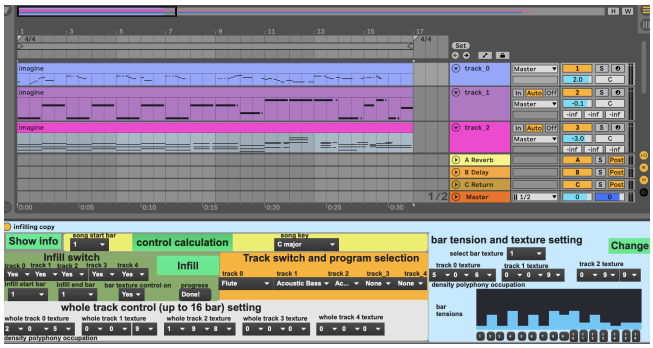

FIGURE 6. The infilling interface as an Ableton Live plugin.

TABLE 11. In total, 24 participants finished the survey, and most are music amateurs. They are evenly distributed in familiarity with AI. The parenthesis lists the percentage of each group.

	Music knowledge	AI familiar
Amateur/No	18 (0.75)	7 (0.292)
Intermediate/Yes	3 (0.125)	9 (0.375)
Expert/Somewhat	3 (0.125)	8 (0.333)

human-composed music. We selected a 16-bar MIDI file with three tracks (melody, bass, and harmony) as a template song. Two AI-generated versions were created by infilling all three tracks using our SMER model and MMM, respectively. This method preserved the original song's structure while replacing the musical content with AI-generated material. Participants were presented with three pairs of songs: 1. Pair One: An AI-generated version (from SMER or MMM) and the original human-composed song. 2. Pair Two: An AI-generated version (from the other model) and the original human-composed song. 3. Pair Three: The two AI-generated versions from SMER and MMM. For each pair, participants were asked to identify which piece they believed was human-composed and which was AI-generated. The results, shown in Table 12 indicate that when an AI-generated piece was paired with the original human-composed music (Pairs One and Two), participants could often distinguish the human-composed music, noting its higher quality. However, when both pieces in a pair were AI-generated (Pair Three), participants found it more challenging to discern differences, suggesting that the AI-generated music had reached a level of quality comparable to human compositions in certain aspects.

Interestingly, Table 13 reveals that participants with higher levels of musical knowledge were better at identifying the AI-composed pieces, particularly in Pair Three. When asked to provide reasons for their choices, some participants described the human-composed music as "smoother," "more natural," and "harmonious." One listener specifically mentioned the presence of "trill notes" in the human-written version, which added to its expressiveness. Some participants mistakenly identified AI-generated songs as human-composed, citing qualities such as "cohesive melody," "smooth transitions," "more off-beat notes," and

"greater variation." Conversely, when participants correctly identified AI-generated pieces, they noted characteristics like "flat melody," "lack of dynamics," "unnatural progression," and "no emotional movement."

The second part of the survey focused on comparing the quality of AI-generated music between our SMER model and MMM. We selected three different 16-bar template songs with three tracks each. Both AI models generated versions for each template song. For each participant, we randomly selected one version from each model for each template song, resulting in three pairs of songs per participant. The order of the songs within each pair was randomized to prevent any ordering bias.

Participants were asked to listen to each pair and indicate which version they preferred, along with reasons for their preference. The results are summarized in Table 14. The findings show that the SMER model was preferred over MMM in two out of the three template songs. Participants praised the SMER-generated music for being "coherent," "melodic," and "comfortable" to listen to. Some criticisms included that the music was "simple" or "lacking excitement." In contrast, listeners found MMM's music to be "complex" and "interesting," but also described it as "messy," "disjointed," or "rough." These comments suggest that while MMM may produce more intricate compositions, they can sometimes lack cohesion and musicality. The SMER model, on the other hand, tends to generate music that is musically sensible and pleasant, even if it may be perceived as less adventurous. Some participants commented on a "robotic sound" in both AI-generated pieces, which could be attributed to the lack of velocity variation in the MIDI files and the use of basic sound synthesis methods. This highlights the importance of incorporating expressive dynamics and more advanced synthesis techniques in future work to enhance the realism of AI-generated music.

VI. ABLETON LIVE PLUGIN

We developed a Max for Live (M4L) plugin to connect the SMER model with the Ableton Live interface. The interface is illustrated in Figure 6, where the upper part shows the arrangement view of the input MIDI, and the lower part displays the M4L plugin.

The plugin provides several parameter selections, including:

- **Section to Infill**: Users can select specific regions of the MIDI track to apply the infilling process.
- **Track-Level and Bar-Level Texture Control Settings**: Adjustments can be made to the texture controls at both the track and bar levels, allowing for fine-grained manipulation of musical attributes.
- **Bar Tonal Tension Setting**: Users can specify the desired tonal tension for each bar, influencing the harmonic complexity of the generated music.

These control parameters are implemented using the `umenu` and `multislider` objects in M4L, providing an intuitive and flexible user experience.

To use the interface, the user follows these steps:

1) **Establish Connection**: Enter the IP address of the model server to establish a connection.
2) **Import MIDI File**: Select and import the input MIDI file into Ableton Live's arrangement view.
3) **Display MIDI Information**: Click the *Show Info* button to display the number of bars and other relevant information about the MIDI file. This data is calculated locally and helps the user understand the structure of the input.
4) **Set Control Parameters**: Adjust the desired control settings using the plugin interface to tailor the generation process.
5) **Trigger Functions**: Use the *Control Calculation* and *Infill* buttons to initiate specific functions on the server side. These functions are configured through the API and perform tasks such as calculating control parameters and generating the infilled music.

The control parameters in this plugin are designed to be easily adaptable to different models. The supplementary material provides a demonstration of this interface for music infilling.

VII. DISCUSSION

A. KEY FINDINGS AND COMPARISONS

1) **SMER vs. REMI:** While SMER is more musically meaningful, it *does not* consistently outperform REMI under *multiple simultaneous* controls, indicating potential trade-offs in complexity and encoding style. However, SMER's explicit rest modeling *does* yield advantages in certain occupation and tension scenarios.
2) **Impact of Bar-Level Textures:** Adding bar-level textures significantly *improves track-level texture control*, especially under the "full" condition. Under the "partial" condition, substituting one texture with an "`unk`" token can actually *enhance* the controllability of the other textures, showcasing a flexible trade-off. Furthermore, in the "prediction" condition, predicting bar-level textures *before* note generation yields *better* overall alignment between local (bar) and global (track) attributes.
3) **Comparison with MMM:** Our SMER-based model produces music that *closely matches* the original control parameters, outperforming MMM in density control. In a user study, SMER was often preferred for its more coherent or "faithful" outputs, although MMM's results were perceived as more novel in some cases.

B. REAL WORLD APPLICATIONS

While our experiments focus on the Lakh MIDI dataset (primarily Western pop), SMER is not bound to a particular style. Its event-based structure—with explicit rests and bar alignment—can extend to other traditions as long as the

TABLE 12. Turing test of AI-generated music. The number is the ratio of user selecting that option. The AI-generated version has a similar quality in pair one and a lower quality in pair two. When two AI-generated versions are in a pair, the result is balanced for the AI/human decision.

Song pair	Real type	Listener's decision					
		AI	Reason	Human	Reason	Not sure	Reason
pair 1	AI 1	0.454	Flat melody Robotic Static and lacks dynamics straight and on the beat	0.409	Simple cohesive melody and backing No apparent break Soft/smoother	0.136	Sound quality seems to be higher than the second
	Human 1	0.272	The accent is abrupt It sounds a bit more ''confusing''	0.5	Beautiful melody Emotional	0.227	If AI, some insertion of syllables is a bit abrupt. If human, there seems to be a 'human error ' when playing piano
pair 2	AI 2	0.695	Not flow ''normal'' sense A few notes not coherent Not natural	0.214	Smooth	0.087	This could have been recorded by a young human player or AI
	Human 2	0.409	Different instruments volume not balanced	0.545	Smoother with fewer drops in sound Natural and flowed More harmonious It has trill notes	0.045	Good continuity, but bass accents still feel mechanical
pair 3	AI 1	0.542	Lack of direction or narrative, meandering Few notes in the wrong key Not much feeling Unnatural rhythm	0.333	More off beat notes Smooth and complete	0.125	
	AI 2	0.416	Very robotic playing Not natural No motion	0.458	More variation Very coherent More natural Comfortable	0.125	Can't tell any difference from song 1 Some unexpected note choices

TABLE 13. In the Turing test, the number of listeners predicting the correct AI/human in each music level category. The parenthesis includes the percentage of that category in that song pair. Pair 3 is the most difficult, and listeners with higher music knowledge perform better.

question	amateur	intermediate	expert
pair 1	5 (0.83)	0	1 (0.17)
pair 2	6 (0.66)	2 (0.22)	1 (0.11)
pair 3	2 (0.5)	1 (0.25)	1 (0.25)

music can be discretized into pitches and bar-based durations. Future research will explore refining the token vocabulary (e.g., for triplets or microtonal systems) and training on diverse corpora (e.g., jazz, classical) to further validate SMER's robustness across genres.

Besides controlled evaluations, we have also explored how the SMER model and its infilling interface operate in real-world creative workflows. The first author used the system for a pop song submission to the AI Song Contest 2022[1], where the infilling approach enabled him to preserve a chosen song structure (e.g., verse, chorus, bridge) while iteratively replacing each track or bar. This iterative process greatly streamlined the co-creation workflow, letting him quickly generate and refine each section without sacrificing overall cohesion.

The model was employed by a professional composer, Mickey Bryan, for the "Music and the Machine" concert at the Tung Auditorium in Liverpool[2]. Bryan used the infilling interface to compose both classical and pop pieces, offering invaluable feedback on real-world usability. For instance, he appreciated how bar-level texture controls helped modify specific bars while maintaining the rest of the composition. However, he also highlighted potential refinements—such as providing a finer pitch-range control and reducing repetitive fragments when the local context is repeated. These insights confirm that our infilling-based framework can seamlessly integrate into professional workflows, albeit with scope for further enhancements (e.g., more granular note-level editing or user-friendly visualizations of generated vs. user-specified controls).

[1]https://www.aisongcontest.com/participants-2022/snagon

[2]https://amlab.liverpool.ac.uk/events/musicandthemachine.html

TABLE 14. The quality comparison between three song pairs generated from three songs as a starting point. Each pair has three versions, and the comments are aggregated. SMER has a better quality for song pairs 1 and 2 and the same quality for pair 3. The main good reason for SMER is "coherent, melodic, comfortable" and the bad reason for "boring, simple". MMM is good for "complex, fun" and bad for "messy, rough".

		SMER	MMM	Same
Song 1	Ratio	0.7	0.22	0.08
	Good reason	Coherent Better melody More interesting Logical Eurhythmic More feel	A clear rhythm Elegant	
	Bad reason	Bored	2 songs cramped together into 1 A mess Incongruent Robotic	
Song 2	Ratio	0.434	0.347	0.217
	Good reason	Had space, relax Clear rhythm There is feel Melodic Comfortable Makes more sense	Interesting mixture Vivid Fun Love the complexity	
	Bad reason	Bored Simple	Too many quick notes Messy Rough	
Song 3	Ratio	0.217	0.217	0.565
	Good reason	Coherent Comfortable Appealing melody Slightly more feel	Slightly easier to listen Good melody Insightful	
	Bad reason	Scratchy Out of tune Boring Simple Not good melody Not enough motifs	Rough melody Needs more motifs	

Overall, these experiences show the practical impact of controllable infilling: it preserves high-level structure yet remains flexible enough for local explorations. This approach can benefit composers seeking a balance between retaining certain musical ideas and exploring novel variations.

C. BROADER IMPLICATIONS OF CONTROLLABILITY IN AI MUSIC GENERATION

The broader implications of controllability in AI music generation extend beyond technical advancements, influencing creativity, accessibility, and ethical considerations in music production.

- **Democratizing Music Creation and Enhancing Accessibility:** Controllable AI models lower the barrier to music composition, enabling non-experts to create complex pieces by adjusting parameters such as genre, mood, or track density. For example, platforms like Suno [35] and open-source models such as YuE [36] and [37] can transform simple text or lyric prompts into full songs, making music creation more accessible to a wider audience.
- **Enhancing Human-AI Collaboration in Creative Workflows:** By integrating controllability, AI shifts from being a passive tool to acting as an active creative collaborator. For example, Suno allows users to specify musical styles, persona of the song besides lyrics. It can also edit part of the generated song, fostering co-creation where human intent guides AI output.
- **Interdisciplinary Applications and Real-World Impact:** The capability to control AI-generated music opens doors for applications in various fields, including gaming, film scoring, and therapeutic music. For instance, sentiment-controllable systems using conditional variational autoencoders [38] can dynamically adjust musical elements to align with narrative emotions, thereby enhancing the audience's experience in films or interactive media.
- **Ethical and Copyright Considerations:** As AI-generated music becomes increasingly controllable and stylistically nuanced, issues of authorship and intellectual property naturally arise. This evolution calls for greater transparency and close collaboration between technology developers and the music industry to ensure ethical practices and protect the rights of creators [39, 40].

VIII. CONCLUSION

In this work, we conducted a comprehensive study on **model controllability in symbolic music generation**, aiming to provide composers and users of AI-generated music with more fine-grained control. Our primary contributions are:

- **SMER Encoding:** We introduced SMER, an event-based symbolic music encoding that uses five basic duration tokens and explicitly represents rests. This design aligns with standard music notation by ensuring that the sum of note and rest durations in a bar exactly matches the bar length, thus enhancing the model's grasp of musical structure.
- **Bar-Level Texture Controls:** To further improve controllability, we incorporated bar-level texture tokens (for density, occupation, and polyphony), allowing users to specify local musical attributes in addition to global track-level controls.

In summary, our results emphasize that *both* a musically aligned encoding (SMER) *and* the introduction of local, bar-level control tokens are key to achieving highly controllable music generation. By balancing global structure with fine-grained edits, we offer a pathway toward more transparent and user-driven AI-assisted composition workflows.

However, this work has several limitations:

1) **Triplet Notes**: The representation of music duration does not include triplet notes. In this work, triplet notes are rounded to the nearest standard duration, which may not accurately capture specific musical ideas that musicians express using triplets.
2) **Sequence Length**: The total input music length is limited to 16 bars. While this is acceptable

for interactive generation scenarios where users can iteratively adapt the generated music, some users may prefer generating whole songs with complex musical structures.

3) **Velocity Information**: Velocity is not included in the music representation, leading to generated music that may sound robotic and lack dynamic expression.

In future work, we aim to address these limitations by incorporating triplet note representations, extending the model to handle longer sequences, and including velocity information to enhance expressiveness. Additionally, we will explore the relationship between generated music quality and the inclusion of more musically meaningful parameters as controls.

IX. SUPPLEMENTARY INFORMATION

The supplement files include the synthesised mp3 files used for the user study. It also includes a demo video of using this model interactively in the Ableton Live software.

APPENDIX A
CONTROL TOKENS

1) **Song-level controls**:
   - **Key**: The song's key can be one of the 24 major and minor keys.
   - **Tempo**: The song's tempo (speed) is categorized into seven bins.
   - **Time Signature**: The time signature of the song, including 4/4, 3/4, 2/4, and 6/8.
2) **Track-level controls**:
   - **Note Density Rate** (D_t):

$$D_t = \frac{n_t}{N_{\text{max}}}$$

where n_t is the number of notes in track t, and $N_{\text{max}} = 16 \times B$ is the maximum number of time steps in the track, with B being the number of bars. The minimum duration is set as a 16th note. Those density levels are represented as d_0 to d_9 (10 discrete categories), and each track has its own density control.

   - **Polyphony Rate** (P_t):

$$P_t = \frac{\tau_{\text{poly}}}{\tau_{\text{note}}}$$

where τ_{poly} is the number of time steps with more than one note played simultaneously (polyphonic notes), and τ_{note} is the total number of time steps with any note. Those polyphony levels are represented as y_0 to y_9 (10 discrete categories), and each track has its own density control.

   - **Occupation Rate** (O_t):

$$O_t = \frac{\tau_{\text{note}}}{N_{\text{max}}}$$

where τ_{note} and N_{max} are as defined above. Those occupation levels are represented as o_0 to o_9 (10 discrete categories), and each track has its own density control.

   - **Instrument**: The MIDI instrument number for the track.
3) **Bar-level controls**:
   - **Tonal Tension**: The tensile strain metric from Herremans and Chew [27] calculates the tonal tension for each bar.
   - **Bar-Level Texture Controls**: The same three types of texture controls (note density rate, polyphony rate, occupation rate) are calculated for each track in each bar. The details are in Section III-D.

REFERENCES

[1] C. A. Huang, A. Vaswani, J. Uszkoreit, I. Simon, C. Hawthorne, N. Shazeer, A. M. Dai, M. D. Hoffman, M. Dinculescu, and D. Eck, "Music transformer: Generating music with long-term structure," in *7th Int. Conf. on Learning Representations*, New Orleans, USA, 2019.

[2] S.-L. Wu and Y.-H. Yang, "Musemorphose: Full-song and fine-grained music style transfer with just one transformer vae," *arXiv preprint arXiv:2105.04090*, 2021.

[3] C.-Z. A. Huang, H. V. Koops, E. Newton-Rex, M. Dinculescu, and C. J. Cai, "AI song contest: Human-ai co-creation in songwriting," *arXiv preprint arXiv:2010.05388*, 2020.

[4] Y.-S. Huang and Y.-H. Yang, "Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions," in *Proc. of the 28th ACM Int. Conf. on Multimedia*, Seattle, USA, 2020, pp. 1180–1188.

[5] J. Ens and P. Pasquier, "Mmm : Exploring conditional multi-track music generation with the transformer," *ArXiv*, vol. abs/2008.06048, 2020.

[6] S. Sulun, M. E. Davies, and P. Viana, "Symbolic music generation conditioned on continuous-valued emotions," *IEEE Access*, vol. 10, pp. 44 617–44 626, 2022.

[7] K. Choi, C. Hawthorne, I. Simon, M. Dinculescu, and J. Engel, "Encoding musical style with transformer autoencoders," in *International Conference on Machine Learning*. PMLR, 2020, pp. 1899–1908.

[8] G. Brunner, Y. Wang, R. Wattenhofer, and S. Zhao, "Symbolic music genre transfer with cyclegan," in *2018 ieee 30th international conference on tools with artificial intelligence (ictai)*. IEEE, 2018, pp. 786–793.

[9] Y.-C. Yeh, W.-Y. Hsiao, S. Fukayama, T. Kitahara, B. Genchel, H.-M. Liu, H.-W. Dong, Y. Chen, T. Leong, and Y.-H. Yang, "Automatic melody harmonization with triad chords: A comparative study," *Journal of New Music Research*, vol. 50, no. 1, pp. 37–51, 2021.

[10] Y.-W. Chen, H.-S. Lee, Y.-H. Chen, and H.-M. Wang,

"Surprisenet: Melody harmonization conditioning on user-controlled surprise contours," *arXiv preprint arXiv:2108.00378*, 2021.

[11] D. von Rütte, L. Biggio, Y. Kilcher, and T. Hoffman, "Figaro: Generating symbolic music with fine-grained artistic control," *arXiv preprint arXiv:2201.10936*, 2022.

[12] B. Genchel, A. Pati, and A. Lerch, "Explicitly conditioned melody generation: A case study with interdependent rnns," *arXiv preprint arXiv:1907.05208*, 2019.

[13] R. Bougueng Tchemeube, J. J. Ens, and P. Pasquier, "Calliope: A co-creative interface for multi-track music generation," in *Creativity and Cognition*, 2022, p. 608–611.

[14] Y. Ren, J. He, X. Tan, T. Qin, Z. Zhao, and T.-Y. Liu, "Popmag: Pop music accompaniment generation," in *Proceedings of the 28th ACM International Conference on Multimedia*, 2020, pp. 1198–1206.

[15] M. E. Malandro, "Composer's assistant: Interactive transformers for multi-track midi infilling," *arXiv preprint arXiv:2301.12525*, 2023.

[16] G. Brunner, A. Konrad, Y. Wang, and R. Wattenhofer, "Midi-vae: Modeling dynamics and instrumentation of music with applications to style transfer," *arXiv preprint arXiv:1809.07600*, 2018.

[17] Y.-Q. Lim, C. S. Chan, and F. Y. Loo, "Clavinet: Generate music with different musical styles," *IEEE MultiMedia*, vol. 28, no. 1, pp. 83–93, 2020.

[18] R. Guo, I. Simpson, T. Magnusson, C. Kiefer, and D. Herremans, "A variational autoencoder for music generation controlled by tonal tension," *Joint Conference on AI Music Creativity (CSMC + MuMe)*, 2020.

[19] H. H. Tan and D. Herremans, "Music fadernets: Controllable music generation based on high-level features via low-level feature modelling," *ISMIR*, 2020.

[20] N. Meade, N. Barreyre, S. C. Lowe, and S. Oore, "Exploring conditioning for generative music systems with human-interpretable controls," *arXiv preprint arXiv:1907.04352*, 2019.

[21] A. Faitas, S. E. Baumann, T. R. Næss, J. Tørresen, and C. P. Martin, "Generating convincing harmony parts with simple long short-term memory networks," in *Proceedings of the International Conference on New Interfaces for Musical Expression*. Universidade Federal do Rio Grande do Sul, 2019, pp. 325–330.

[22] Z. Guo, D. Makris, and D. Herremans, "Hierarchical recurrent neural networks for conditional melody generation with long-term structure," in *2021 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2021, pp. 1–8.

[23] R. Guo, I. Simpson, C. Kiefer, T. Magnusson, and D. Herremans, "Musiac: An extensible generative framework for music infilling applications with multi-level control," in *Artificial Intelligence in Music, Sound, Art and Design: 11th International Conference, EvoMUSART 2022, Held as Part of EvoStar 2022, Madrid, Spain, April 20–22, 2022, Proceedings*. Springer, 2022, pp. 341–356.

[24] G. Hadjeres and L. Crestel, "The piano inpainting application," *arXiv preprint arXiv:2107.05944*, 2021.

[25] A. Vaswani, N. M. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *arXiv:1706.03762*, 2017.

[26] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "Ctrl: A conditional transformer language model for controllable generation," *arXiv*, 2019.

[27] D. Herremans and E. Chew, "Tension ribbons: Quantifying and visualising tonal tension," in *2nd International Conference on Technologies for Music Notation and Representation (TENOR)*, Cambridge, UK, 2016.

[28] M. Gotham, K. Gullings, and C. Hamm, *Open Music Theory*. Oklahoma State University, 2022.

[29] E. Chew, "The spiral array: An algorithm for determining key boundaries," in *International Conference on Music and Artificial Intelligence*. Springer, 2002, pp. 18–31.

[30] D.-V.-T. Le, L. Bigo, M. Keller, and D. Herremans, "Natural language processing methods for symbolic music generation and information retrieval: a survey," *arXiv*, vol. 2402.17467, 2024.

[31] S. Oore, I. Simon, S. Dieleman, D. Eck, and K. Simonyan, "This time with feeling: Learning expressive musical performance," *Neural Computing and Applications*, vol. 32, no. 4, pp. 955–967, 2020.

[32] C. Walshaw, "Abc notation," https://abcnotation.com/about, 2022.

[33] A. Fan, M. Lewis, and Y. Dauphin, "Hierarchical neural story generation," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*. Melbourne, Australia: Association for Computational Linguistics, Jul. 2018, pp. 889–898.

[34] R. Guo, D. Herremans, and T. Magnusson, "Midi Miner – A Python library for tonal tension and track classification," *arXiv:1910.02049*, Oct. 2019.

[35] Suno, Inc., "Suno: Ai music generator," https://suno.com/, accessed: February 3, 2025.

[36] R. Yuan, H. Lin, S. Guo, G. Zhang, and et al., "Yue: Open music foundation models for full-song generation," https://github.com/multimodal-art-projection/YuE, 2025, gitHub repository.

[37] K. Bhandari, A. Roy, K. Wang, G. Puri, S. Colton, and D. Herremans, "Text2midi: Generating symbolic music from captions," *arXiv preprint arXiv:2412.16526*, 2024.

[38] J. Min, Z. Gao, L. Wang, Z. Cai, H. Wu, and A. Zhang, "A sentiment-controllable music generation system based on conditional variational autoencoder," in *2024 International Symposium on Electrical, Electronics and*

Information Engineering (ISEEIE), 2024, pp. 352–357.

[39] R. Drury, "You're not supposed to launder my music! music as data in the training of generative ai music models," in *Innovation in Music: Adjusting Perspectives*, 1st ed. Focal Press, 2024, ch. 4.

[40] M. Wei, M. Modrzejewski, A. Sivaraman, and D. Herremans, "Prevailing research areas for music ai in the era of foundation models," *arXiv preprint arXiv:2409.09378*, 2024.

RUI GUO received a Ph.D. in Music Technology from the University of Sussex, Brighton, UK. His current research interests include large language models (LLMs) and knowledge graphs applied to medical data. This work was conducted as part of his Ph.D. research.

DORIEN HERREMANS is an associate professor at Singapore University of Technology and Design, where she leads the Audio, Music, and AI (AMAAI) Lab. Before being at SUTD, she was a Marie Sklodowska-Curie Postdoctoral Fellow at the Centre for Digital Music at Queen Mary University of London. Prof. Herremans has been a pioneer in the music technology field, publishing her first generative music model 20 years ago. Her work has been published in over 100 top tier publications including AAAI, Information Fusion and Expert Systems with Applications to name a few. She was also nominated on the Singapore 100 Women in Tech list in 2021, and shortlisted as one of the top 30 SAIL Award (Super AI Leader) Finalists at the World AI Conference in 2024.

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3554648